

Gebze Technical University
Department of Computer Engineering
CSE 654/484: Natural Language Processing
Fall 2024

Homework 2

200104004039

Sümeyye Zeynep Gürbüz

Turkish Question Answering System for Gebze Technical University Official Student Rules

➤ Project Overview

This project aims to build a Question Answering (QA) system that can extract relevant answers from Gebze Technical University's official student rules document, as provided on the university's website (<https://www.gtu.edu.tr/icerik/1479/592/lisans-yonetmelik-ve-yonergeler.aspx>). Students will work on implementing a natural language understanding system specifically tailored for university rules and regulations in Turkish.

The project will involve preprocessing official documents, training or fine-tuning a Turkish language transformer-based model, and deploying a system capable of answering natural language questions about the rules.

➤ Project Objectives

1. Parse and preprocess the Gebze Technical University student rules document to create a QA dataset.
2. Build a QA system to answer Turkish questions about university regulations.

3. Fine-tune a pre-trained Turkish transformer model for extracting precise answers.
4. Create a user-friendly interface where students can ask questions and receive answers directly from the rules.

Modules Used in the Project

Below is a list of modules used in the project, along with their purpose in building the Turkish Question Answering (QA) system.

1. datasets

- a. *Purpose*: Used to load and manage datasets for natural language processing tasks.
- b. *Key Functions*:
 - i. **load_dataset**: Loads and processes the dataset from various formats like JSON, CSV, or directly from an online source.
 - ii. **Dataset**: Provides functionality to manipulate datasets (e.g., filtering, mapping).

2. transformers

- a. *Purpose*: Provides pre-trained transformer models and utilities for natural language tasks.
- b. *Key Components*:
 - i. **AutoTokenizer**: Automatically loads a tokenizer compatible with a specified transformer model.
 - ii. **AutoModelForQuestionAnswering**: Loads a transformer model pre-trained for QA tasks.
 - iii. **Trainer**: Simplifies the training process of transformer models.
 - iv. **TrainingArguments**: Configures hyperparameters for model training.
 - v. **DefaultDataCollator**: Handles batching and padding of input data during training.
 - vi. **pipeline**: Provides an easy-to-use interface for running tasks such as question answering.

3. sklearn.metrics.pairwise

- a. *Purpose*: Calculates pairwise similarity metrics.
- b. *Key Function*:
 - i. **cosine_similarity**: Measures similarity between vectors, commonly used for comparing text embeddings.

4. **fuzzywuzzy**

- a. *Purpose*: Implements string matching and similarity scoring.
- b. *Key Functionality*:
 - i. **fuzz**: Contains methods for computing ratios of similarity between strings.

5. **tqdm**

- a. *Purpose*: Provides a progress bar for loops, useful for tracking long-running processes.

6. **torch**

- a. *Purpose*: A deep learning library for tensor computations and building machine learning models.
- b. *Key Components*:
 - i. **torch.nn.functional**: Contains various functions for deep learning operations (e.g., softmax).
 - ii. **torch.Tensor**: Basic data structure for storing and manipulating multidimensional arrays.

7. **numpy**

- a. *Purpose*: A fundamental library for numerical computations in Python.
- b. *Key Uses*:
 - i. Handling numerical arrays.
 - ii. Performing mathematical operations on tensors and embeddings.

8. **re**

- a. *Purpose*: Provides regular expression matching operations for text preprocessing.

9. **string**

- a. *Purpose*: Contains constants and utilities for string manipulation, such as removing punctuation.

10. **math**

- a. *Purpose*: Provides mathematical functions like logarithms and trigonometric operations used in advanced computations.

These modules together provide the necessary tools for preprocessing text, fine-tuning models, and implementing the QA system effectively.

Section 1: Class Implementations

- **SimpleContextRetriever**

This class is designed to retrieve the most relevant contexts (text passages) for a given question, combining both semantic and traditional text-matching techniques like BM25. Below is a detailed breakdown:

1. Class Components

i. (**`__init__`**)

- **Purpose:** Initializes the retriever with a custom embedding retriever function and defines default parameters like batch size.
- **Attributes:**
 - a. `retriever`: A callable that computes embeddings for a batch of text inputs.
 - b. `batch_size`: Controls the number of contexts processed at once during embedding calculations.
 - c. `contexts` and `context_embeddings`: Store preprocessed contexts and their embeddings, respectively.

ii. **`normalize_text`**

- **Functionality:** Cleans and normalizes input text by:
 - a. Converting to lowercase.
 - b. Removing leading/trailing spaces.
 - c. Replacing multiple spaces with a single space.
- **Usage:** Ensures uniformity in text processing for both contexts and queries.

iii. **`calculate_bm25_scores`**

- **Functionality:** Implements the BM25 algorithm for relevance scoring.
 - a. Tokenizes query and documents.
 - b. Calculates Inverse Document Frequency (IDF) for query terms.
 - c. Computes scores for each document based on term frequency and document length.
- **Usage:** Provides a traditional IR-based scoring mechanism for matching questions with contexts.

iv. `prepare_contexts`

- **Functionality:** Prepares and embeds contexts for retrieval:
 - a. Extracts and normalizes contexts from a given dataset.
 - b. Removes duplicate contexts.
 - c. Generates embeddings for all contexts using the retriever.
 - d. Normalizes embeddings to ensure consistency in similarity computations.
- **Usage:** This method must be called before performing context retrieval to set up the data.

v. `find_best_contexts`

- **Functionality:** Finds the most relevant contexts for a given question:
 - a. Normalizes the question and generates its embedding.
 - b. Computes semantic similarity between the question and all context embeddings.
 - c. Combines semantic similarity scores with BM25 scores using a weighted formula.
 - d. Returns the top k contexts based on combined scores.
- **Usage:** Used to retrieve multiple high-scoring contexts with their relevance scores.

vi. `find_best_context`

- **Functionality:** Finds the single best context for a question:
 - a. Calls `find_best_contexts` to get the top k contexts.
 - b. Combines them into a single string, respecting a `max_tokens` limit.
- **Usage:** Provides a concise and optimized context for downstream tasks like question answering.

2. Implementation

i. **Hybrid Retrieval:** Combines two retrieval mechanisms:

- **Semantic Matching:** Uses contextual embeddings from the retriever to calculate cosine similarity.
- **Traditional IR:** Employs BM25 for scoring based on term frequency and document length.

- ii. **Normalization:** Both context embeddings and query embeddings are normalized to ensure that similarity scores are scale-invariant.
- iii. **Batch Processing:** Handles context embedding in batches to optimize memory usage.

3. Usage in the System

i. Preprocessing Phase:

→ The `prepare_contexts` method processes the dataset and generates embeddings for all contexts.

ii. Query Phase:

- A question is input into `find_best_context` or `find_best_contexts`, and the method outputs the most relevant text passage(s).
- These contexts are passed to a QA model to extract precise answers.

- **QATrainer**

This class extends the `Trainer` class from the Hugging Face Transformers library. It customizes the behavior of loss computation to allow for potential flexibility during training, such as returning outputs along with the loss.

1. Class Components

i. `(__init__)`

- Inherits the constructor from the `Trainer` class, so there is no explicit redefinition in this class.
- All initialization parameters like model, dataset, and training arguments are handled by the base class.

ii. `compute_loss`

- **Purpose:** Customizes the loss computation during training.
- **Parameters:**
 - a. `model`: The model being trained (e.g., a transformer model for question answering).
 - b. `inputs`: The batch of input data, typically including the input IDs, attention masks, and labels.
 - c. `return_outputs` (optional): If `True`, returns the model's outputs along with the loss. Default is `False`.

- d. `num_items_in_batch` (optional): Unused in this implementation but could be used for advanced batching strategies.

→ **Implementation:**

- a. Calls the model with the input data (`model(**inputs)`).
- b. Retrieves the loss value from the model's outputs.
- c. Conditionally includes the outputs if `return_outputs` is `True`.

→ **Return Value:**

- a. If `return_outputs` is `True`, returns a tuple (`loss`, `outputs`).
- b. Otherwise, only the loss is returned.

2. Usage in the QA System

i. **Customized Training Loop:**

- By overriding the `compute_loss` method, the class enables additional flexibility, such as analyzing the outputs during training without modifying the main Trainer loop.

ii. **Integration with Hugging Face:**

- Extends the base Trainer class, making it compatible with existing tools and APIs in the Hugging Face library.
- This allows seamless usage of features like distributed training, evaluation, and logging.

3. Advantages of Customizing `compute_loss`

i. **Modularity:**

- Keeps the loss computation logic separate from other training configurations.

ii. **Flexibility:**

- Enables advanced debugging or monitoring of model outputs during training.
- Can be further extended to handle different loss functions or custom metrics.

iii. **Scalability:**

- The unused `num_items_in_batch` parameter provides a placeholder for future enhancements, such as batch-level adjustments.

Section 2: Standalone Functions

1. load_data

- a. **Purpose:** Loads the dataset from a specified JSON file for the QA system.
- b. **Implementation:**
 - i. Uses the `load_dataset` function from the datasets library to read a JSON file containing QA data.
 - ii. File path is specified as `"/content/QAs.json"`.
- c. **Return Value:** A dataset object containing the loaded QA data.
- d. **Usage:** Used in the preprocessing stage to read the dataset into memory.

2. setup_system

- a. **Purpose:** Sets up the foundational components of the QA system, including the model, tokenizer, and retriever.
- b. **Parameters:**
 - i. `model_name` (default: `"dbmdz/bert-base-turkish-cased"`): Specifies the pre-trained transformer model to use.
- c. **Implementation:**
 - i. Detects the available device (CUDA or CPU) using PyTorch.
 - ii. Loads the pre-trained model and tokenizer using the transformers library.
 - iii. Initializes a pipeline for feature extraction (used for semantic similarity).
 - iv. Transfers the model to the specified device for efficient computation.
- d. **Return Value:** A tuple containing:
 - i. `model`: Pre-trained QA model.
 - ii. `tokenizer`: Tokenizer for pre-processing input text.
 - iii. `retriever`: Feature extraction pipeline for context retrieval.
 - iv. `device`: The computation device.
- e. **Usage:** Sets up the core components required for training and inference.

3. realign_dataset

Cleans, filters, and restructures the dataset to align the QA data for better training and evaluation.

- a. **Implementation:**

- a. **Initialization:** Creates a new dictionary to hold cleaned QA pairs (question, context, answer).
- b. **Iterates through the dataset:**
 - i. Strips whitespace and validates that the question, context, and answer are non-empty.
 - ii. Removes prefixes like "evet," or "hayır," from the answers.
 - iii. Calculates the overlap between the cleaned answer and context words to find the best matching context window.
- c. **Filters:**
 - i. Excludes items with insufficient overlap between the context and answer.
 - ii. Adjusts the answer to match the best context window if overlap exceeds 50%.
- d. **Handles Errors:** Catches and logs exceptions for problematic entries.
- e. **Return Value:** A dictionary containing the filtered and realigned data:
 - i. Keys: question, context, answer.
 - ii. Values: Lists of cleaned and structured strings.
- f. **Usage:** Prepares the dataset for model fine-tuning or evaluation by ensuring high-quality QA pairs.

4. preprocess_data

Prepares data for training a question-answering model by tokenizing and aligning inputs, including start and end positions of the answers within their contexts.

1. Implementation

- a. **Initialization:**
 - i. Extracts and cleans questions, contexts, and answers from the dataset.
 - ii. Initializes an all_inputs dictionary to store tokenized data and position mappings.
- b. **Text Matching:**
 - i. Uses the fuzz library to calculate a similarity ratio between the answer and substrings of the context.
 - ii. Identifies the best match with a similarity ratio exceeding 85% and records its starting position.

- c. **Tokenization:**
 - i. Applies the tokenizer to the question and context:
 - ii. Supports truncation (only `_second` for context truncation).
 - iii. Uses sliding windows (via `stride`) to handle long contexts.
 - iv. Returns offset mappings and overflow information.
- d. **Alignment of Answer Positions:**
 - i. Maps the character-level start and end positions of the answer to token-level indices:
 - ii. Ensures the indices fall within the tokenized context window.
 - iii. If alignment fails, assigns start and end positions as 0.
- e. **Data Aggregation:**
 - i. Collects tokenized data (`input_ids`, `attention_mask`) and aligned positions (`start_positions`, `end_positions`) for all splits.
 - ii. Stores overflow mappings to track the original sample index.
- f. **Conversion to Torch Tensors:**
 - i. Converts lists into PyTorch tensors for compatibility with training pipelines.
 - ii. Logs warnings if critical data (e.g., `input_ids`) is missing.

2. Return Value

- a. Returns a dictionary with tokenized inputs and answer positions:
 - i. **`input_ids`**: Tokenized inputs.
 - ii. **`attention_mask`**: Attention masks for each input.
 - iii. **`start_positions`**: Token-level start positions of the answer.
 - iv. **`end_positions`**: Token-level end positions of the answer.
 - v. **`overflow_to_sample_mapping`**: Mapping of splits to original samples.

3. Usage

- a. This function is essential for:
 - i. Preparing datasets for fine-tuning QA models like BERT.
 - ii. Ensuring that model inputs are properly aligned with answer positions.
 - iii. Handling long contexts by using sliding windows.

5. `fine_tune_model`

Fine-tunes a QA model on a dataset by:

- Preprocessing questions to retrieve relevant contexts.

- Preparing tokenized datasets for training and evaluation.
- Training the model using specified hyperparameters.
- Returning the trained model wrapped in a custom trainer for further use.

1. Implementation

a. Preprocessing Questions with Context:

i. Defines `preprocess_with_context`:

1. Uses `context_retriever.find_best_context()` to retrieve a relevant context for each question.
2. Adds the retrieved context to the dataset.
3. Tokenizes and aligns the data using `preprocess_data`.

b. Tokenized Dataset Mapping:

- i. Applies the preprocessing function to the entire dataset using `dataset.map()`.
- ii. Removes columns no longer needed after preprocessing (e.g., raw questions, contexts).

c. Filtering Empty Examples:

- i. Filters out entries where the `input_ids` are empty to ensure valid examples for training.
- ii. Logs progress via the `desc` parameter of the `filter()` method.

d. Validation:

- i. Ensures the processed dataset is not empty; raises an error if no valid data remains after filtering.

e. Training Arguments:

i. Configures TrainingArguments:

1. Specifies learning rate, batch sizes, number of epochs, weight decay, and logging steps.
2. Enables evaluation and model saving after every epoch.
3. Loads the best model at the end based on the "f1" metric.

f. Training Setup:

i. Initializes the QATrainer:

1. Links the model, tokenizer, and processed datasets.
2. Sets a custom metric function (`compute_metrics`) for evaluation during training.

g. Training Execution:

- i. Calls **trainer.train()** to start the fine-tuning process.

6. Return Value:

- a. Returns the trained QATrainer instance, allowing further evaluation or saving of the model.

7. Key Parameters

- a. **model**: The QA model (e.g., BERT) to be fine-tuned.
- b. **tokenizer**: Tokenizer corresponding to the model.
- c. **dataset**: Dataset containing QA pairs.
- d. **context_retriever**: Custom retriever for finding relevant contexts.
- e. **output_dir**: Directory to save model outputs, logs, and checkpoints.

8. Strengths

- a. Automates end-to-end fine-tuning.
- b. Ensures quality inputs by integrating preprocessing and filtering.
- c. Logs progress and ensures reproducibility with saved models and metrics.

9. Usage

- a. Ideal for quickly fine-tuning QA models on custom datasets, especially when contexts need to be dynamically retrieved or aligned.

- **normalize_answer(s):**

- a. Converts text to lowercase, removes punctuation, fixes whitespace, and handles Turkish-specific cases (e.g., $I \rightarrow i$, $\dot{I} \rightarrow i$).
- b. Ensures predictions and ground truths are comparable despite formatting differences.

- **compute_em(prediction, truth):**

- a. Computes Exact Match (EM) by comparing normalized prediction and truth.
- b. Returns 1 for exact matches, otherwise 0.

- **compute_f1(prediction, truth):**

- a. Tokenizes predictions and truths after normalization.
- b. Calculates precision, recall, and F1 score for overlapping tokens.
- c. Handles edge cases (e.g., empty predictions or truths).

- **compute_metrics(eval_pred, tokenizer, dataset):**

- a. Converts model logits into text predictions using `tokenizer.decode`.
- b. Compares predictions to the ground truths from dataset.

- c. Outputs averaged EM and F1 scores.
- **train_and_run_model():**
 - a. **System Setup:**
 - i. Loads the model, tokenizer, retriever, and device using `setup_system()`.
 - b. **Data Preparation:**
 - i. Loads raw data with `load_data()`.
 - ii. Prepares relevant contexts with `context_retriever`.
 - iii. Aligns QA pairs using `realign_dataset`.
 - c. **Dataset Structuring:**
 - i. Converts the aligned dataset into a Dataset object.
 - ii. Splits the data into training and test sets (80% train, 20% test).
 - d. **Model Fine-Tuning:**
 - i. Uses `fine_tune_model` to preprocess, train, and validate the model.
 - e. **Interactive QA:**
 - i. Runs `ask_question` for querying the trained model.
- **ask_question**

Function to set up to interactively ask questions, retrieve the best matching context, and use the fine-tuned model to provide an answer.

a. **Workflow Breakdown**

- i. **User Input:**
 - The function asks the user for a question.
 - If the user types "çıkış", it exits the loop.
- ii. **Context Retrieval:**
 - It uses the `context_retriever.find_best_context(question)` to find the most relevant context for the given question.
 - If no context is found, it prompts the user to be more specific.
- iii. **Tokenization:**
 - The question and context are tokenized and prepared for input into the model.

- Special tokens are added, and the sequence is padded/truncated to the maximum length of 512 tokens.
- iv. **Model Prediction:**
- The model generates logits for the start and end positions of the answer within the context.
 - Softmax is applied to get probabilities for both start and end positions.
 - The start and end indices are derived by selecting the maximum probabilities.
- v. **Answer Extraction:**
- The answer is extracted by slicing the tokenized input using the predicted start and end indices.
 - The answer tokens are decoded into a readable string.
- vi. **Output:**
- The context and the extracted answer are printed.
- vii. If the answer is empty, it indicates no valid answer was found.

OUTPUTS

```
trainer = QATrainer(  
[44/44 01:55, Epoch 4/4]
```

Epoch	Training Loss	Validation Loss	Em	F1
1	3.502200	0.261477	0.000000	0.000000
2	0.181000	0.002613	0.000000	0.000000
3	0.005200	0.000678	0.000000	0.000000
4	0.001700	0.000549	0.000000	0.000000

Çıkmak için 'çıkış' yazın.

Soru:

<

Epoch	Training Loss	Validation Loss	Em	F1
1	3.918000	0.499719	0.000000	0.000000
2	0.371400	0.006315	0.000000	0.000000
3	0.013200	0.001219	0.000000	0.000000
4	0.003500	0.000869	0.000000	0.000000

Çıkmak için 'çıkış' yazın.

Soru: Ders bilgi formu nedir ve neyi içerir?

Context: devam şartını yerine getirmiş ancak ders bilgi formunda belirtilen yükümlülükleri yerine getirmeyen öğ

Cevap: ##n

Soru: Devam durumu nasıl takip edilir?

Context: öliys sonuçlarına göre alınan puanlar harf notlarına dönüştürülür ve başarı durumu aşağıdaki tablodaki

Cevap: ##na

Soru: